FFmpeg Filtering Guide

FFmpeg has access to many filters and more are added on a regular basis. To see what filters are available with your build see `ffmpeg -filters`.

Contents

- Documentation
- Filter syntax
  - Filtergraph, Chain, Filter relationship
  - Escaping characters
- Examples
  - Scaling
  - Speed up your video
  - Multiple input overlay in 2x2 grid
  - Burnt in Timecode
  - Synthetic Input
  - Other Filter Examples
- Filter Metadata
- Scripting Filters
  - From a File
  - With a Shell Script
- Developing your own Filters
- Visualizing filters

Documentation

Refer to the FFmpeg filters documentation for more information and examples for each filter. This wiki page is for user contributed examples and tips, and contributions to this page are encouraged.

Filter syntax

When documentation refers to "filter options", or says the "filter accepts the following options", the syntax to use for the options is as described in **FFmpeg Filters**, section *4.1 Filtergraph syntax*. Briefly, that syntax is to add, after the name of the filter, an `=` character, then the name of the first filter option, an `=` character, and the value of that option. If you want to specify further filter options, you delimit them with `:`, then append the name of the next filter option, an `=` character, and the next value.

For instance, to apply the loudnorm filter to an audio stream, the basic syntax is:

```
ffmpeg -i input -filter:a loudnorm  output
```

To add the `print_format` and `linear` filter options, use the syntax:

```
ffmpeg -i input -filter:a loudnorm=print_format=summary:linear=true  output
```

As described in section *4.1 Filtergraph syntax*, you can omit the option names and `=` characters, supplying just the values, delimited by `:` characters. For example, in this invocation:

```
ffmpeg -i input -vf scale=iw/2:-1 output
```

FFmpeg assumes the option names in the order they are declared in the source code. For instance, in this use of the scale filter, FFmpeg assumes the option name `width` for the value `iw/2`, and the name `height` for the value `-1`.

Filtergraph, Chain, Filter relationship

What follows the `-vf` in an ffmpeg command line is a filtergraph description. This filtergraph may contain a number of chains, each of which may contain a number of filters.

Whilst a full filtergraph description can be complicated, it is possible to simplify it for simpler graphs provided ambiguity is avoided.

Remembering that filters in a chain are separated by commas "," chains by a semicolon ";" and that if an input or output is not specified it is assumed to come from the preceding or sent to the following item in the chain.

The following are equivalent:

```
ffmpeg -i input -vf [in]scale=iw/2:-1[out] output
ffmpeg -i input -vf scale=iw/2:-1 output                                    #
```

As are:

```
ffmpeg -i input -vf [in]yadif=0:0:0[middle];[middle]scale=iw/2:-1[out] output #
ffmpeg -i input -vf [in]yadif=0:0:0,scale=iw/2:-1[out] output                 #
ffmpeg -i input -vf yadif=0:0:0,scale=iw/2:-1  output                         #
```

Escaping characters

As described in the documentation, it can be necessary to escape commas "," that need to appear in some arguments, for example the select filter:

```
ffmpeg -i input -vf select='eq(pict_type\,I)' -vsync vfr output_%04d.png
```

However an alternative, which also allows for white space within the filtergraph, and which may assist in clarity of reading complex graphs, is to enclose the whole filtergraph within double quotes " " thus:

```
ffmpeg -i input -vf "select='eq(pict_type,I)'" output       # to select only I f
ffmpeg -i input -vf "yadif=0:-1:0, scale=iw/2:-1" output    # deinterlace then r
```

Note that the examples given in the documentation mix and match the use of "full quoting" and "\" escaping, and that use of unusual shells may upset escaping. See Notes on filtergraph escaping for more information.

Examples

Scaling

Starting with something simple. Resize a 640x480 input to a 320x240 output.

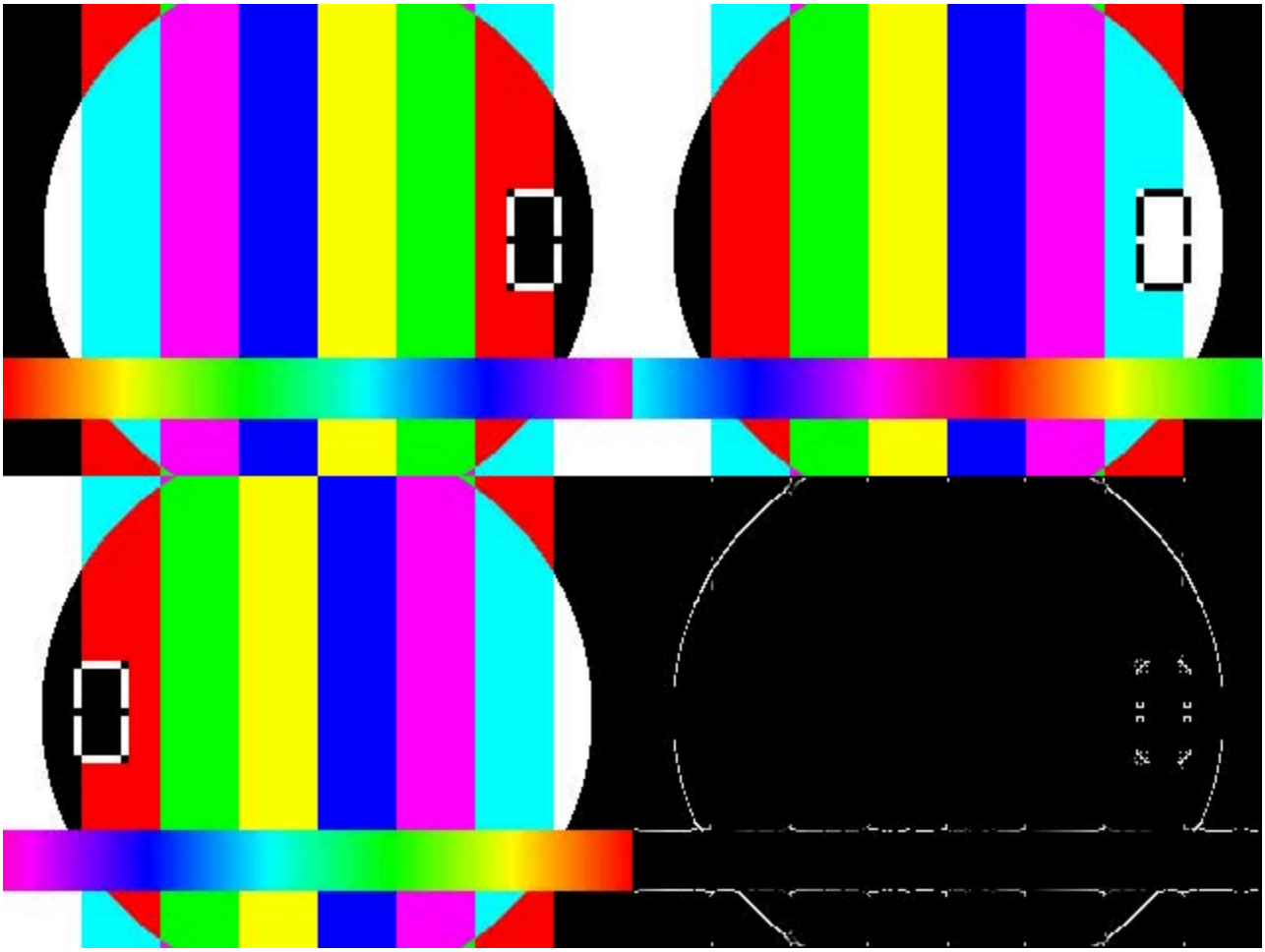
```
ffmpeg -i input -vf scale=iw/2:-1 output
```

`iw` is input width. In this example the input width is 640. 640/2 = 320. The `-1` tells the scale filter to preserve the aspect ratio of the output, so in this example the scale filter will choose a value of 240. See the FFmpeg documentation for additional information.

Speed up your video

See How to speed up / slow down a video for examples.

Multiple input overlay in 2x2 grid



Here four inputs are filtered together using the `-filter_complex` option. In this case all of the inputs are the `-f lavfi -i testsrc` (the testsrc source filter) but could be other inputs. (Encoded video is 5 seconds and uses the 'ffv1' video codec.)

Within the filtergraph the first input is unchanged, and the other three inputs are individually filtered using hflip, negate, and edgedetect. The hstack and vstack filters are then used to stack each video into the desired location.

```
ffmpeg \
  -f lavfi -i testsrc   -f lavfi -i testsrc \
  -f lavfi -i testsrc   -f lavfi -i testsrc \
  -filter_complex \
"[1:v] negate     [a]; \
 [2:v] hflip      [b]; \
 [3:v] edgedetect [c]; \
 [0:v] [a]      hstack=inputs=2 [top]; \
 [b]   [c]      hstack=inputs=2 [bottom]; \
 [top] [bottom] vstack=inputs=2 [out]" \
  -map "[out]" \
  -c:v ffv1   -t 5   multiple_input_grid.avi
```

This next example is the same as above, but uses the pad and overlay filters instead. The pad filter is used to make an appropriate sized background, and overlay is used to place each video into the correct location. This method is slower than using hstack + vstack as shown above.

```
ffmpeg \
  -f lavfi -i testsrc   -f lavfi -i testsrc \
  -f lavfi -i testsrc   -f lavfi -i testsrc \
  -filter_complex \
filter_complex \
"[0:v]  pad=iw*2:ih*2 [a]; \
 [1:v]  negate        [b]; \
 [2:v]  hflip         [c]; \
 [3:v]  edgedetect    [d]; \
 [a] [b] overlay=w   [x]; \
 [x] [c] overlay=0:h [y]; \
 [y] [d] overlay=w:h [out]" \
  -map "[out]" \
  -c:v ffv1   -t 5   multiple_input_grid.avi
```

Be aware that frames are taken from each input video in timestamp order, so it is a good idea to pass all overlay inputs through a `setpts=PTS-STARTPTS` filter to have them begin in the same zero timestamp, such as `[0:v]hflip,setpts=PTS-STARTPTS[a];[1:v]setpts=PTS-STARTPTS[b];[a][b]overlay`.

Burnt in Timecode

Using the drawtext video filter.

PAL 25 fps non drop frame:

```
ffmpeg -i in.mp4 -vf "drawtext=fontfile=/usr/share/fonts/truetype/DroidSans.ttf:
x=(w-tw)/2: y=h-(2*lh): fontcolor=white: box=1: boxcolor=0x00000000@1" -an -y ou
```

NTSC 30 fps drop frame

```
(change the : to a ; before the frame count)_____________________________

ffmpeg -i in.mp4 -vf "drawtext=fontfile=/usr/share/fonts/truetype/DroidSans.ttf:
x=(w-tw)/2: y=h-(2*lh): fontcolor=white: box=1: boxcolor=0x00000000@1" -an -y ou
```

Synthetic Input

The testsrc source filter generates a test video pattern showing a color pattern, a scrolling gradient, and a timestamp. This is useful for testing purposes.

This example will create a 10 second output, 30 fps (300 frames total), with a frame size of 1280x720:

```
ffmpeg -f lavfi -i testsrc=duration=10:size=1280x720:rate=30 output.mpg
```

`ffplay` can also be used to view the resulting filtergraph:

```
ffplay -f lavfi -i "testsrc=duration=10:size=1280x720:rate=30"
```

You can also specify testsrc as a filter:

```
ffmpeg -filter_complex testsrc OUTPUT
```

Another type of testsrc is using the smptebars source filter:

```
ffmpeg -f lavfi -i "smptebars=duration=5:size=1280x720:rate=30" output.mp4
```

Or a color

```
./ffmpeg -f lavfi -i color=c=red:size=100x100
```

There are other options for generating synthetic video input, see here and here ("generic equation" filter).

Other Filter Examples

- Fancy Filtering Examples – Examples for various psychedelic effects and other weird filtering.
- Null describes the nullsink filter.

Filter Metadata

Filters may write metadata entries that can be used for debugging their functionality, or extracting additional information from the input file. For a general description, see the metadata filter documentation.

Metadata keys are defined by the respective filters, and there is no globally accessible list. Some of them are mentioned in the filter documentation itself.

Here are some important metadata keys, all starting with `lavfi.<filter>`, where `<filter>` is the name of the filter itself.

Filter	Key	Description
`aphasemeter`	`phase`	
`astats`	`Bit_depth2`	
`astats`	`Bit_depth`	
`astats`	`Crest_factor`	
`astats`	`DC_offset`	
`astats`	`Dynamic_range`	
`astats`	`Flat_factor`	
`astats`	`Max_difference`	
`astats`	`Max_level`	
`astats`	`Mean_difference`	
`astats`	`Min_difference`	
`astats`	`Min_level`	
`astats`	`Overall.Bit_depth2`	
`astats`	`Overall.Bit_depth`	
`astats`	`Overall.DC_offset`	
`astats`	`Overall.Flat_factor`	
`astats`	`Overall.Max_difference`	
`astats`	`Overall.Max_level`	
`astats`	`Overall.Mean_difference`	
`astats`	`Overall.Min_difference`	
`astats`	`Overall.Min_level`	
`astats`	`Overall.Number_of_samples`	
`astats`	`Overall.Peak_count`	
`astats`	`Overall.Peak_level`	
`astats`	`Overall.RMS_difference`	
`astats`	`Overall.RMS_level`	
`astats`	`Overall.RMS_peak`	

astats	Overall.RMS_trough	
astats	Peak_count	
astats	Peak_level	
astats	RMS_difference	
astats	RMS_level	
astats	RMS_peak	
astats	RMS_trough	
astats	Zero_crossings_rate	
astats	Zero_crossings	
bbox	h	
bbox	w	
bbox	x1	
bbox	x2	
bbox	y1	
bbox	y2	
blackframe	pblack	
cropdetect	h	
cropdetect	w	
cropdetect	x1	
cropdetect	x2	
cropdetect	x	
cropdetect	y1	
cropdetect	y2	
cropdetect	y	
ebur128	r128.I	
ebur128	r128.LRA.high	
ebur128	r128.LRA.low	
ebur128	r128.LRA	
ebur128	r128.M	
ebur128	r128.S	
freezedetect	freeze_duration	Duration of the freeze period
freezedetect	freeze_end	End of the freeze period
freezedetect	freeze_start	Start of the freeze period
psnr	mse.u	
psnr	mse.v	
psnr	mse.y	
psnr	mse_avg	
psnr	psnr.u	

psnr	psnr.v	
psnr	psnr.y	
psnr	psnr_avg	
signalstats	HUEAVG	
signalstats	HUEMED	
signalstats	key	
signalstats	SATAVG	
signalstats	SATHIGH	
signalstats	SATLOW	
signalstats	SATMAX	
signalstats	SATMIN	
signalstats	UAVG	
signalstats	UBITDEPTH	
signalstats	UDIF	
signalstats	UHIGH	
signalstats	ULOW	
signalstats	UMAX	
signalstats	UMIN	
signalstats	VAVG	
signalstats	VBITDEPTH	
signalstats	VDIF	
signalstats	VHIGH	
signalstats	VLOW	
signalstats	VMAX	
signalstats	VMIN	
signalstats	YAVG	
signalstats	YBITDEPTH	
signalstats	YDIF	
signalstats	YHIGH	
signalstats	YLOW	
signalstats	YMAX	
signalstats	YMIN	
silencedetect	silence_duration	Duration of the silence period
silencedetect	silence_end	End of the silence period
silencedetect	silence_start	Start of the silence period
ssim	All	
ssim	dB	
ssim	u	

`ssim`	`v`	
`ssim`	`y`	

Scripting Filters

From a File

Suppose you want to change some filter parameters based on an input file. Some filters support the option to receive commands via `sendcmd`. Run `ffmpeg -filters` and check the `C` column – if it is present, a filter supports receiving input this way.

For example, if you want to rotate the input at seconds 0, 1, and 2, create a file called `cmd.txt` with the content:

```
0 rotate angle '45*PI/180';
1 rotate angle '90*PI/180';
2 rotate angle '180*PI/180';
```

Now run an example stream:

```
ffmpeg -f lavfi -i testsrc -filter_complex "[0:v]sendcmd=f=cmd.txt,rotate" -f ma
```

Some examples of this technique can be seen here.

With a Shell Script

When building complex filtergraphs, it may help to break the command down into manageable pieces. However one needs to be careful when joining them all together, in order to avoid issues due escaped characters.

The following example shows a sample bash script containing a filtergraph of one chain with three filters: yadif, scale and drawtext.

```
#!/bin/bash
# ffmpeg test script

path="/path/to/file/"

in_file="in.mp4"
out_file="out.mp4"

cd $path

filter="yadif=0:-1:0, scale=400:226, drawtext=fontfile=/usr/share/fonts/truetype
text='tod- %X':x=(w-text_w)/2:y=H-60 :fontcolor=white :box=1:boxcolor=0x00000000
```

```
codec="-vcodec libx264  -pix_fmt yuv420p -b:v 700k -r 25 -maxrate 700k -bufsize

command_line=(ffmpeg -i "$in_file" -vf "$filter" "$codec" -an "$out_file")

echo "${command_line[@]}"
"${command_line[@]}"
```

Note that the filtergraph spans more than one line. The echo command shows the full command as it is executed. This is useful for debugging.

The array in the `$command_line` variable helps avoid loss of the quotes which occurs otherwise. Other shells may behave differently.

Developing your own Filters

- See FFmpeg filter HOWTO

Visualizing filters

Sometimes `ffmpeg` inserts different filters (ex: pixel format conversion filters) into a filter graph to make it work. To see what is "actually going on" run it like `ffmpeg -loglevel debug ...`. Useful for finding auto inserted filters.

You can also get a fun graphical representation of your filtergraph by running the "included" graph2dot command like `graph2dot "my filter graph"`.

There is also a `dumpgraph` option if you can convert to `lavfi` syntax: `ffmpeg -dumpgraph 1 -f lavfi -i "..."` which outputs a nice ASCII art view of your graph.

▶ **Attachments** (1)

Last modified on Dec 28, 2023, 10:50:21 PM

t a g s
filtering